

DQN: 픽셀에서 행동 가치까지

Human-level control through deep reinforcement learning

Volodymyr Mnih et al. (2015)

화면 관측, experience replay, target snapshot으로 구성된 Atari 학습

화면 관측과 숨은 게임 상태

에이전트는 emulator 내부 상태나 사람이 설계한 feature를 직접 받지 않는다

구분	에이전트가 받는 정보	직접 알 수 없는 정보
화면	게임의 픽셀 영상	물체의 속도와 숨은 변수
행동	현재 게임에서 허용한 action 집합	행동 뒤의 장기 결과
보상	점수 변화로 정의한 한 단계 reward	미래 할인 누적 보상
상태 표현	최근 화면을 전처리한 짧은 history	emulator의 완전한 내부 상태

한 화면만으로 운동 방향을 알기 어려워 짧은 frame history가 필요하다

DQN이 추정하는 Q 값

$Q(w, a)$ 는 관측 history w 에서 행동 a 를 한 뒤 기대하는 할인 누적 보상의 추정치다

기호	뜻	구분
w	network가 받는 상태 history 표현	emulator의 숨은 상태와 다름
a	현재 선택할 수 있는 행동	행동의 승리 확률이 아님
r	한 transition에서 관측한 즉시 보상	Q 전체와 같지 않음
γ	미래 보상을 줄여 합산하는 할인율	이 논문의 설정은 0.99
θ, θ^-	online 파라미터와 과거 복사본	두 개의 독립 학습기가 아님

정책은 Q 추정치를 참고해 행동을 고르며 Q 자체는 확률이 아니다

네 장의 frame history

원본 화면을 그대로 한 장 넣지 않고 flicker와 운동 정보를 고려해 입력을 구성한다

- | | | |
|----|------------|--|
| 01 | 연속 화면 | 현재 frame과 바로 이전 frame을 함께 본다 |
| ↓ | | |
| 02 | Flicker 완화 | 두 frame의 같은 위치에서 더 큰 pixel 값을 취한다 |
| ↓ | | |
| 03 | 전처리 | luminance를 사용하고 84×84 로 줄인다 |
| ↓ | | |
| 04 | History 입력 | 최근 네 장을 쌓아 $84 \times 84 \times 4$ tensor를 만든다 |

네 장의 입력은 짧은 운동 단서를 제공하지만 완전한 game history는 아니다

한 번의 forward pass와 행동별 Q 벡터

DQN은 다음 행동의 확률이 아니라 모든 허용 행동의 Q 추정치를 한꺼번에 출력한다



보고된 게임마다 허용 행동 수는 4개에서 18개이며 output 수도 이에 맞춘다

신경망 Q-learning의 불안정성 압력

online trajectory, 바뀌는 정책, bootstrap target이 서로 영향을 준다

압력	발생 이유	학습에 미치는 위험
연속 표본의 상관	이웃 frame과 transition이 같은 trajectory에서 이어짐	gradient가 비슷한 방향으로 치우칠 수 있음
데이터 분포 변화	Q가 바뀌면 행동 정책과 이후 수집 경험도 바뀜	최근 경험에 따라 학습 분포가 급히 이동함
움직이는 target	현재 추정값이 다음 상태의 bootstrap 값에도 영향을 줌	한 update가 예측과 target을 함께 움직일 수 있음

DQN은 replay와 target snapshot으로 서로 다른 두 압력을 완화한다

안정화 장치 1 : uniform experience replay

현재 순서대로만 학습하지 않고 유한한 과거 transition 집합에서 균등하게 표본화한다

- | | | |
|----|---------------|---|
| 01 | 상호작용 | 현재 w 에서 a 를 수행하고 r 과 다음 w 를 관측한다 |
| ↓ | | |
| 02 | Transition 저장 | w , a , r , 다음 w 를 유한 replay memory에 추가한다 |
| ↓ | | |
| 03 | 균등 표본화 | 저장된 transition에서 random minibatch를 뽑는다 |
| ↓ | | |
| 04 | 반복 학습 | 같은 경험이 서로 다른 update에 다시 쓰일 수 있다 |
| ↓ | | |
| 05 | 오래된 항목 교체 | 용량을 넘으면 이전 transition이 memory에서 빠진다 |

이 논문의 replay는 중요도를 학습하는 prioritized replay가 아니다

안정화 장치 2: 고정된 target snapshot

online theta를 주기적으로 복사하고 복사 사이의 C번 update 동안 theta-를 고정한다

01	Online theta	minibatch loss의 gradient로 계속 갱신된다
↓		
02	주기적 복사	매 C번 update마다 theta를 theta-에 복사한다
↓		
03	Target theta-	다음 C번 update 동안 같은 snapshot을 유지한다
↓		
04	Bootstrap 값	다음 상태 행동들의 최댓값을 theta-에서 계산한다
↓		
05	지연된 영향	작은 online 변화가 target에 즉시 반영되지 않는다

gradient는 online theta를 바꾸며 target theta-는 복사 시점에만 바뀐다

DQN target과 종료 분기

비종료 전이: $y = r + \gamma \max Q(w_{\text{next}}, a_{\text{next}}; \theta^-)$ 를 사용한다

전이 유형	학습 target y	이유
비종료	$r + \gamma \max Q(w_{\text{next}}, a_{\text{next}}; \theta^-)$	다음 상태 이후의 할인 return을 bootstrap
종료	r	episode가 끝나므로 다음 상태 return을 더하지 않음
현재 예측	$Q(w, a; \theta)$	실제로 수행했던 행동 a 의 online 추정
오차	$y - Q(w, a; \theta)$	online θ 를 갱신할 방향을 제공

y 는 관측 정답이 아니라 고정 snapshot을 포함한 bootstrap 추정치다

교육용 target 계산

가정: $r = 1$, $\gamma = 0.99$, 비종료 다음 상태의 target Q 벡터는 $[2.0, 3.5, 3.0]$ 이다

항목	값	의미
다음 상태 Target Q	left 2.0, stay 3.5, right 3.0	theta-의 행동별 값
최대 행동과 값	stay, $\max = 3.5$	argmax와 max의 결과
비종료 target y	$1 + 0.99 \times 3.5 = 4.465$	y는 4.465로 유지
현재 Q와 raw TD	$Q = 2.0$, raw TD = 2.465	Error clip 적용값 1
종료 target y	1	Bootstrap 항 없음

Target은 4.465이고 raw error 2.465의 update 영향은 1로 clip한다

한 minibatch update의 데이터 흐름

과거 transition을 표본화한 뒤 target snapshot과 online network의 역할을 분리한다

01	Replay 표본	과거 transition w, a, r , 다음 w 를 무작위로 뽑는다
↓		
02	고정 target	θ^- 에서 다음 상태 최대 Q 를 구해 y 를 만든다
↓		
03	현재 예측	online θ 에서 $Q(w, a)$ 를 계산한다
↓		
04	TD loss	y 와 online 예측의 차이를 loss로 계산한다
↓		
05	Online update	error를 clip한 gradient로 θ 만 바꾸고 θ^- 는 유지한다

Error clipping은 gradient 영향을 제한하지만 target y 와 target θ^- 를 바꾸지 않는다

DQN network 구조

Convolutional feature extractor와 linear output이 행동별 Q 값을 계산한다

단계	구성	출력의 역할
입력	$84 \times 84 \times 4$ frame history	짧은 시간 정보가 포함된 상태 표현
Conv 1	32 filters, 8×8 , stride 4	화면의 넓은 지역 pattern 추출
Conv 2	64 filters, 4×4 , stride 2	더 작은 spatial pattern 결합
Conv 3	64 filters, 3×3 , stride 1	상위 표현 계산
FC와 output	512 units, linear outputs 4-18개	각 허용 행동의 $Q(w, a)$ 추정

마지막 output은 policy 확률이 아니라 행동마다 하나씩 대응하는 Q 추정치다

49개 게임에 적용한 학습 설정

논문은 게임마다 별도 network를 학습하되 같은 architecture와 hyperparameter를 사용했다

항목	설정	해석
주요 학습량	게임마다 5천만 frame	약 38일 game experience
Replay memory	Capacity 100만	오래된 항목 교체
Update 표본	minibatch 32, gamma 0.99	uniform replay 표본
탐색	첫 100만 frame에 epsilon 1.0에서 0.1	이후 0.1 유지
환경 처리	action repeat 4, reward를 -1, 0, 1로 clipping	보상 크기 차이를 잃음

Reward clipping은 공통 설정을 돕지만 각 게임의 보상 크기 정보를 보존하지 않는다

공통 학습 recipe와 게임별 network

49개 게임에서 같은 방법을 시험했지만 하나의 network가 모든 게임을 동시에 학습한 것은 아니다

구성	49개 게임에서 공유	게임마다 별도
Architecture	같은 convolutional 구조	허용 행동 수에 맞춘 output
Algorithm	같은 replay와 target-network 절차	각 게임의 경험으로 update
Hyperparameter	같은 기본 학습 설정	별도의 학습 run 수행
평가 목적	게임별 조정 없이 일반적인 recipe를 시험	게임별 점수와 기준선을 비교

결과는 공통 recipe의 범용성을 시험하며 하나의 multi-game policy를 증명하지 않는다

DQN, human, random의 평가 조건

Human-normalized score는 서로 다른 세 기준의 평균 점수를 사용하므로 측정 절차가 해석에 포함된다

평가 대상	실행 조건	점수의 역할
DQN agent	게임당 30회, 서로 다른 no-op 시작, 최대 5분, epsilon 0.05	학습된 agent score
Human tester	약 2시간 연습 뒤 약 20 episode, 최대 5분, audio와 pause 없음	human 기준 점수
Random agent	10 Hz마다 random action을 고르고 그 사이에 반복	0% 기준 점수
Fig. 3 error bar	30개 agent 평가 episode의 standard deviation	평가 episode 사이 변동

No-op 시작은 같은 초기 trajectory만 반복하는 평가를 피하기 위한 조건이다

Human-normalized score 읽는 법

$$\text{점수} = 100 \times (\text{DQN} - \text{random}) / (\text{human} - \text{random})$$

정규화 값	기준	뜻하지 않는 것
0%	random agent와 같은 score	실패 확률 100%
75%	random과 human 기준 차이의 75%	승률 75%
100%	해당 human tester와 같은 score	완벽한 플레이
100% 초과	해당 조건에서 human 기준보다 높은 score	모든 사람보다 우수함

이 백분율은 성공 확률이 아니라 random과 한 human 기준 사이의 선형 위치다

49개 게임의 주요 결과

보고된 수치는 공통 recipe를 게임별로 학습하고 정해진 baseline과 비교한 결과다

보고 결과	분모와 조건	올바른 해석
43개 게임	비교 가능한 49개 중	당시 인용한 기존 RL 방법의 최고 점수보다 높음
29개 게임	49개 중 human-normalized 75% 이상	논문이 사용한 human-level operational threshold 충족
같은 기본 설정	49개 game별 별도 network	게임마다 algorithm이나 feature를 조정하지 않음

29/49는 human을 이긴 게임 수가 아니라 75% 이상인 게임 수다

Figure 3이 보여 주는 범위

정해진 평가 조건의 game score를 비교하며 모든 게임에서 같은 수준을 증명하지는 않는다

관찰할 수 있는 내용

게임별 DQN, 기존 agent, human 기준의 정규화 score

49개 게임 중 29개가 75% 이상

같은 recipe가 여러 게임에서 경쟁력 있는 결과를 냄

게임별 성능 차이와 30회 평가 변동이 존재함

확대할 수 없는 주장

환경이 달라도 동일한 절대 능력을 측정함

29개 게임에서 human score 100%를 넘음

하나의 network가 49개 게임을 동시에 학습함

모든 game과 시작 상태에서 같은 성능을 보장함

Human-level이라는 표현은 논문의 score normalization과 75% threshold 안에서 읽어야 한다

Replay와 target network의 component evidence

Extended Data Table 3은 주요 5천만-frame 결과와 다른 1천만-frame ablation 조건을 사용한다

항목	Ablation 설정	해석 범위
구성 조합	Replay on/off와 별도 target on/off의 모든 조합	Component 효과 비교
Learning rate	세 가지 값을 각 조합에 적용	단일 설정 비교가 아님
학습량	1천만 frame	Main 5천만보다 짧음
검증 주기	25만 frame마다 13만 5천 validation frame	여러 학습 시점 평가
보고값	최고 평균 episode score, 5분 제한 없음	Main score와 직접 비교 불가

Ablation은 두 component의 유용성을 지지하지만 보편적 안정성 보장은 아니다

DQN의 핵심과 후속 질문

짧은 화면 history를 행동 가치로 바꾸고 replay와 고정 target으로 학습 흐름을 안정화한다

- | | | |
|----|-----------------|---|
| 01 | 상태 표현 | 전처리한 최근 네 frame으로 짧은 history를 구성한다 |
| ↓ | | |
| 02 | 행동 가치 | 한 forward pass에서 모든 허용 행동의 Q를 추정한다 |
| ↓ | | |
| 03 | Replay | 과거 transition을 균등 표본화해 반복 학습한다 |
| ↓ | | |
| 04 | Target snapshot | 과거 theta-로 bootstrap target의 이동을 늦춘다 |
| ↓ | | |
| 05 | 후속 Double DQN | online theta가 고르고 target theta-가 같은 행동의 Q를 평가한다 |

DQN은 representation, replay, target 계산을 하나의 학습 system으로 결합했다